BLOQUE 1

Python

Sintaxis, tipos y control de flujo.

bloque1.py

```
entorno = "venv + VS Code"
version = 3.12

for sesion in range(1, 5):
    print(f"Sesion {sesion}: lista")
```

CONTEXTO

¿Por qué Python es el lenguaje de la IA?

01 Ecosistema

NumPy, pandas, scikit-learn, PyTorch, TensorFlow. La comunidad científica publica en Python.

02 Lenguaje pegamento

Sintaxis sencilla por fuera, C y CUDA por dentro. Prototipas rápido sin pagar el rendimiento.

03 Curva de entrada

Poco código ceremonial: el foco está en el modelo y en los datos, no en la sintaxis.

04 Interactividad

REPL y notebooks: exploras datos y ves resultados sin ciclo de compilación.

05 Estándar de facto

Casi toda API de IA (OpenAI, Hugging Face, cloud) ofrece SDK en Python antes que en nada más.

06 Y sus límites

Lento en bucles puros, GIL y tipado dinámico. Por eso el cálculo pesado se delega en librerías.

1 Entorno y primer programa

Cómo se ejecuta un .py, cómo se aísla un proyecto y qué aspecto tiene el código bien escrito.

EJECUCIÓN

Interpretado, no compilado (del todo)

CPython traduce a bytecode y lo ejecuta en una máquina virtual.

- No hay paso explícito de compilación: editas y ejecutas.
- Los errores de tipo aparecen en ejecución, no antes.
- Tres formas de ejecutar: script, REPL interactivo y notebook.
- Un fichero .py es a la vez programa y módulo importable.

terminal

```
$ python hola.py        # script
$ python                # REPL
>>> 2 ** 10
1024

# Jupyter: celdas ejecutables
# ideal para explorar datos
```

ENTORNO

Proyecto aislado en 4 órdenes

Todo proyecto de IA empieza igual: entorno virtual propio y dependencias declaradas.

terminal

```
python -m venv .venv                  # crea el entorno en la carpeta .venv
source .venv/bin/activate             # Linux / macOS
.venv\Scripts\activate                # Windows (PowerShell o CMD)

pip install numpy pandas jupyter      # instala solo en este entorno
pip freeze > requirements.txt         # congela versiones para reproducir

python -m pip --version               # comprobar qué pip estamos usando
```

Regla de oro: sin entorno virtual acabas con versiones incompatibles entre proyectos. En IA, reproducir una versión exacta de una librería es parte del trabajo.

SINTAXIS

Lo mismo que ya sabes, con menos ruido

Java / C#

Suma.java

```java
public class Suma {
  public static void main(String[] a) {
    int total = 0;
    for (int i = 1; i <= 10; i++) {
      if (i % 2 == 0) {
        total += i;
      }
    }
    System.out.println(total);
  }
}
```

Python

suma.py

```python
total = 0

for i in range(1, 11):
    if i % 2 == 0:
        total += i

print(total)
```

Idea clave: la indentación no es estilo, es sintaxis. Los dos puntos abren bloque y la sangría (4 espacios) lo delimita. No hay llaves ni punto y coma.

CONVENCIONES

PEP 8: el estilo es parte del lenguaje

La comunidad escribe igual. Tu editor puede aplicarlo solo con black o ruff.

Elemento	Convención	Ejemplo
Variables y funciones	`snake_case`	`tasa_error, cargar_datos()`
Clases	`PascalCase`	`ClasificadorKNN`
Constantes	`MAYÚSCULAS`	`MAX_ITER = 100`
Privado por convenio	`guion bajo inicial`	`_cache, _normalizar()`
Indentación	`4 espacios`	`    if x > 0:`
Longitud de línea	`79-99 caracteres`	`# ruff format`
Comentarios	`explican el porqué`	`# evitamos división por cero`

2

Tipos, variables y operadores

Tipado dinámico pero fuerte: no declaras el tipo, pero el tipo existe y manda.

VARIABLES

Las variables son etiquetas, no cajas

Una variable no reserva espacio: apunta a un objeto que vive en memoria.

- No se declara el tipo; se deduce del valor asignado.
- Una misma variable puede apuntar después a otro tipo.
- type() dice el tipo; id() dice a qué objeto apunta.
- Los type hints son opcionales: documentan, no obligan.

variables.py

```
x = 42              # int
x = "cuarenta"      # ahora str: legal

print(type(x))      # <class 'str'>

a = [1, 2, 3]
b = a               # misma lista
b.append(4)
print(a)            # [1, 2, 3, 4]

# Type hints (opcionales)
tasa: float = 0.05
```

TIPOS BÁSICOS

Cinco tipos que cubren el 90 % del código

Tipo	Ejemplo	Qué conviene recordar
int	42, -7, 10_000	Precisión ilimitada: no hay desbordamiento.
float	3.14, 2e-3	Coma flotante IEEE 754: 0.1 + 0.2 != 0.3.
bool	True, False	Es un int por debajo: True + True == 2.
str	"hola", 'hola'	Inmutable y Unicode. Comillas simples o dobles.
NoneType	None	Ausencia de valor. Se compara con "is None".

OPERADORES

Aritmética: dos divisiones y una potencia

operadores.py

```python
7 / 2       # 3.5   división real: SIEMPRE devuelve float
7 // 2      # 3     división entera (hacia abajo)
-7 // 2     # -4    cuidado: redondea hacia menos infinito
7 % 2       # 1     resto
2 ** 10     # 1024  potencia (no existe Math.pow por defecto)

contador = 0
contador += 1        # existe += -= *= /= //= **= %=
# contador++         # NO existe el incremento en Python

5 < x < 10           # comparaciones encadenadas: legal y legible
not (a and b) or c   # and / or / not en vez de && || !
```

Ojo: and y or devuelven uno de los operandos, no un booleano: "" or "valor" devuelve "valor". Se usa mucho para valores por defecto.

TEXTO

Cadenas y f-strings

El texto es inmutable: cada operación devuelve una cadena nueva.

- f-string: la forma moderna de interpolar y formatear.
- Métodos encadenables: .strip(), .lower(), .split().
- Formato numérico dentro de la propia llave: {x:.2f}.
- Triple comilla para texto multilínea y docstrings.

cadenas.py

```
nombre = "Ada"
acierto = 0.93412

print(f"{nombre}: {acierto:.2%}")
# Ada: 93.41%

print(f"{acierto:.3f}")    # 0.934

linea = "  1,2,3  "
print(linea.strip().split(","))
# ['1', '2', '3']

texto = """varias
lineas"""
```

Ancho, alineación y relleno

Patrón: {valor:[relleno][alineación][signo][ancho][,][.precisión][tipo]}. Aquí, s = "IA" y n = 42.

Formato	Resultado	Significado
`f"|{s:10}|"`	`|IA        |`	Ancho mínimo de 10 caracteres (el texto se alinea a la izquierda)
`f"|{s:>10}|"`	`|        IA|`	> alinea a la derecha
`f"|{s:^10}|"`	`|    IA    |`	^ centra
`f"|{n:<6}|"`	`|42    |`	< alinea a la izquierda (los números van a la derecha por defecto)
`f"|{s:*^10}|"`	`|****IA****|`	El carácter antes de la alineación es el relleno
`f"{n:05d}"`	`00042`	Rellena con ceros hasta 5 cifras (d = entero decimal)

Formato de números

Con x = 3.14159, p = 0.9341, g = 1234567.891 y n = 42.

Formato	Resultado	Significado
`f"{x:.2f}"`	`3.14`	Número fijo de decimales (f = coma fija)
`f"|{x:8.3f}|"`	`|   3.142|`	Ancho total 8 con 3 decimales: alinea columnas de cifras
`f"{x:+.1f}"`	`+3.1`	Muestra siempre el signo, también en positivos
`f"{p:.1%}"`	`93.4%`	Porcentaje: multiplica por 100 y añade el símbolo
`f"{g:,.2f}"`	`1,234,567.89`	Separador de miles con coma
`f"{g:_.0f}"`	`1_234_568`	Separador de miles con guion bajo
`f"{g:.2e}"`	`1.23e+06`	Notación científica (útil en tasas de aprendizaje)
`f"{n:b} {n:o} {n:x}"`	`101010 52 2a`	Binario, octal y hexadecimal

Trucos que ahorran tiempo

fstrings.py

```python
k, acierto = 5, 0.9341
print(f"{k=}, {acierto=:.3f}")        # k=5, acierto=0.934   depuración (3.8+)

modelo = "knn"
print(f"{modelo!r}")                   # 'knn'   !r usa repr(): se ven las comillas
print(f"{modelo.upper()} con k={k * 2}")   # admite expresiones y llamadas

ancho, dec = 10, 2
print(f"{acierto:>{ancho}.{dec}%}")    # formato dinámico con llaves anidadas

print(f"{{literal}} = {k}")            # {literal} = 5   llaves dobles para escribirlas

from datetime import date
print(f"{date.today():%d/%m/%Y}")      # 21/09/2026   también formatea fechas
```

Contexto: los f-strings existen desde Python 3.6. En código antiguo y en muchos tutoriales verás "%.2f" % x y "{:.2f}".format(x): usan el mismo minilenguaje de formato.

CASTING

Convertir de un tipo a otro

Cada tipo tiene una función con su nombre que intenta la conversión. Si no puede, lanza una excepción.

Función	Ejemplos	Resultado	Cuidado con
`int()`	`int("42")  int(3.99)  int("ff", 16)`	`42  3  255`	Trunca; int("3.5") da ValueError
`float()`	`float("3.5")  float(7)  float("1e-3")`	`3.5  7.0  0.001`	float("3,5") da ValueError
`str()`	`str(42)  str(0.1)  str(None)`	`"42"  "0.1"  "None"`	Nunca falla: todo tiene texto
`bool()`	`bool(0)  bool("")  bool("False")`	`False  False  True`	Solo lo vacío o cero es False
`list()`	`list("abc")  list(range(3))`	`['a','b','c']  [0,1,2]`	Acepta cualquier iterable
`tuple() set()`	`tuple([1, 2])  set([1, 1, 2])`	`(1, 2)  {1, 2}`	set() elimina duplicados
`dict()`	`dict([("a", 1), ("b", 2)])`	`{'a': 1, 'b': 2}`	Necesita pares clave-valor

CASTING

Implícito solo entre números; el resto, explícito

casting.py

```python
# Implícito: Python promociona al tipo numérico "más amplio"
3 + 2.5              # 5.5    int + float -> float
True + 1             # 2      bool -> int
7 / 2                # 3.5    / siempre devuelve float

# Explícito: cualquier otra mezcla la conviertes tú
edad = int(input("Edad: "))    # input() SIEMPRE devuelve str
"3" + 4              # TypeError: no concatena str con int
"3" + str(4)         # "34"
float("3,5")         # ValueError: el decimal va con punto
int(float("3.7"))    # 3    de texto decimal a entero, en dos pasos
round(3.7)           # 4    redondear no es lo mismo que convertir
isinstance(edad, int)          # comprobar el tipo antes de operar
```

Comparación: JavaScript convertiría "3" + 4 en "34" sin avisar. Python prefiere fallar pronto y ruidosamente: el traceback dice la línea y los tipos implicados.

3 Control de flujo

Condicionales y bucles: pocas novedades conceptuales, varias sintácticas.

CONDICIONALES

if / elif / else y la verdad implícita

Todo objeto tiene un valor de verdad: vacío o cero es falso.

- elif sustituye a "else if"; no hay switch clásico.
- Falsos: 0, 0.0, "", [], {}, set(), None, False.
- Compara con None usando "is", no "==".
- Existe el condicional en una línea (operador ternario).

condicionales.py

```
nota = 7.5

if nota >= 9:
    print("Sobresaliente")
elif nota >= 5:
    print("Aprobado")
else:
    print("Suspenso")

datos = []
if not datos:
    print("lista vacia")

estado = "ok" if nota >= 5 else "ko"
```

BUCLES

for recorre elementos, no índices

El for de Python es un "for each": itera sobre cualquier iterable.

- range(inicio, fin, paso): fin no se incluye.
- enumerate() da índice y valor a la vez.
- while para condiciones; for para colecciones.
- El bucle else se ejecuta si no hubo break.

bucles.py

```python
for x in [10, 20, 30]:
    print(x)

for i in range(3):          # 0, 1, 2
    print(i)

for i, v in enumerate(["a", "b"]):
    print(i, v)             # 0 a / 1 b

intentos = 0
while intentos < 3:
    intentos += 1

for c in "IA":
    print(c)                # I / A
```

RANGE

range(inicio, fin, paso)

Mismo criterio que el slicing: el inicio se incluye y el final nunca.

Llamada	Valores generados	Qué hay que ver
`range(5)`	`0 1 2 3 4`	Con un argumento es el final; empieza en 0
`range(2, 6)`	`2 3 4 5`	Inicio incluido, final excluido
`range(1, 11)`	`1 2 3 ... 10`	Para llegar hasta N, el final es N + 1
`range(0, 20, 5)`	`0 5 10 15`	El tercer argumento es el paso
`range(10, 0, -3)`	`10 7 4 1`	Paso negativo: cuenta hacia atrás (inicio > final)
`range(5, 2)`	`(ninguno)`	Inicio ≥ final con paso positivo: vacío, sin error
`range(0, 1, 0.1)`	`TypeError`	Solo enteros; con decimales se usa numpy.arange
`range(0, 10, 0)`	`ValueError`	El paso no puede ser cero

RANGE

range no es una lista: calcula, no almacena

range_perezoso.py

```
r = range(0, 1_000_000_000)     # no reserva mil millones de números
import sys
sys.getsizeof(r)                # 48 bytes: guarda solo inicio, fin y paso

len(r)                          # 1000000000
r[10], r[-1]                    # (10, 999999999)   se puede indexar
500 in r                        # True, instantáneo: lo calcula, no lo busca
r[::2]                          # range(0, 1000000000, 2)  slicing -> otro range

list(range(5))                  # [0, 1, 2, 3, 4]   materializar si hace falta
list(reversed(range(5)))        # [4, 3, 2, 1, 0]
sum(range(1, 101))              # 5050

for i in range(len(notas)):     # funciona, pero casi nunca es lo mejor:
    print(i, notas[i])          # mejor enumerate(notas)
```

Idea clave: range genera cada valor cuando se le pide (evaluación perezosa). La misma idea reaparece con generadores, ficheros y datasets que no caben en memoria.

CONTROL FINO

break, continue y el else del bucle

flujo.py

```python
numeros = [4, 8, 15, 16, 23, 42]
objetivo = 15

for n in numeros:
    if n % 2 != 0:
        continue          # salta al siguiente elemento
    if n == objetivo:
        print("encontrado")
        break             # sale del bucle
else:
    print("no encontrado")  # solo si NO hubo break

# match (Python 3.10+): alternativa legible a cadenas de elif
match modelo:
    case "knn" | "vecinos": print("clasificador por distancia")
    case "arbol":           print("arbol de decision")
    case _:                 print("modelo desconocido")
```

Detalle propio: el for...else es exclusivo de Python y sustituye a la típica variable "encontrado = False". Se lee como "si terminó sin encontrar nada".

PRÁCTICA ·

Ejercicios del bloque 1

- E1.1 · Conversor de unidades: pide grados Celsius y muestra Fahrenheit y Kelvin con 2 decimales.
- E1.2 · Clasificador de notas: pide una nota, valida el rango 0-10 y muestra la calificación.
- E1.3 · Tabla de multiplicar con formato alineado usando f-strings.
- E1.4 · FizzBuzz del 1 al 100, en 6 líneas o menos.
- E1.5 · Adivina el número: bucle while con intentos limitados y pistas mayor/menor.
- E1.6 · Sucesión de Fibonacci hasta N usando asignación múltiple a, b = b, a + b.

e1_1.py

```python
# E1.1  esqueleto
c = float(input("Grados Celsius: "))

f = ...
k = ...

print(f"{c:.2f} C = {f:.2f} F")
print(f"{c:.2f} C = {k:.2f} K")
```

Un fichero por ejercicio

PRÁCTICA ·

Ejercicios: tipos, formato y range

- E1.7 · Conversor seguro: pide un valor e indica si es entero, decimal o no numérico, convirtiéndolo al tipo adecuado.
- E1.8 · Ticket de compra: nombre alineado a la izquierda, unidades centradas y precio a la derecha con 2 decimales.
- E1.9 · Con range: múltiplos de 7 hasta 100, cuenta atrás de 10 a 0 y suma de los pares hasta 1000.
- E1.10 · Tabla de -20 °C a 40 °C de 5 en 5 con su equivalente en Fahrenheit, en dos columnas alineadas.

e1_8.py

```python
# E1.8  esqueleto
compra = [("Teclado", 1, 24.9),
          ("Raton", 2, 12.5),
          ("Monitor", 1, 189.0)]

for nombre, uds, precio in compra:
    print(f"{nombre:<12}{uds:^6}{precio:>10.2f}")

# Teclado       1        24.90
```